

CSCE 605 COMPILER DESIGN · MODULE 01 · LECTURE NOTES

What a Compiler Is

A series of representations, each closer to the machine.

MODULE OUTCOMES

- Name the phases and say what each one establishes.
- Explain why the intermediate representation is the central design decision.
- Distinguish compilation from interpretation, and locate the hybrids.
- Explain why a miscompile is uniquely expensive.
- Set up the toolchain this course needs.

1 The phases

Phase	Transformation	What it establishes
Lexical analysis	Characters → tokens.	The vocabulary is valid. Every piece of the input is a recognisable word of the language.
Parsing	Tokens → syntax tree.	The structure is valid. The words are arranged into well-formed phrases.
Semantic analysis	Tree → annotated tree; names resolved, types checked.	The program <i>means</i> something. Every name refers to something, and every operation is applied to things it can apply to.
IR generation	Tree → intermediate representation.	A machine-independent, analysable form. The front end ends here.
Optimisation	IR → better IR.	Nothing. It is the only phase that establishes no new property — its whole obligation is to <i>preserve</i> every property the earlier phases established, while changing the program.
Instruction selection	IR → target instructions.	The operations are ones the machine has. The back end.
Register allocation	Unbounded virtual registers → the machine's finite set.	The program fits the register file, with spills where it does not (Module 11).
Scheduling	Instructions reordered.	The order suits the pipeline (CSCE 614). Must not change meaning.

Framing the phases by what each establishes is more useful than listing them in order, because it explains why they are in that order: each depends on the guarantee the previous one made. And it isolates

optimisation as the odd phase — **the only one whose correctness condition is that it changes nothing observable**, which is exactly why it is the hardest to get right and the source of the worst bugs (§4).

THE FRONT END / BACK END SPLIT IS ABOUT REUSE

The front end depends on the source language and not on the target machine. The back end depends on the machine and not on the language. Neither needs to know anything about the other's domain.

So m languages and n targets require $m + n$ components rather than $m \times n$, provided they all meet at a common intermediate representation. With five languages and six targets that is eleven pieces instead of thirty.

That is the entire reason LLVM exists, and it is why Rust, Swift, Julia, Clang, and a long list of others share a back end despite having almost nothing in common at the source level.

And it is why the IR is the real design decision. It is the interface between the two halves, and interfaces are far harder to change than implementations — every consumer depends on them. LLVM IR has evolved, but its fundamental shape has not, because it cannot.

2 The intermediate representation

THE IR DETERMINES WHICH OPTIMISATIONS ARE EXPRESSIBLE

An optimisation is a question plus a rewrite. 'Is this value the same constant at every use?' — then substitute it. 'Is this computation already available?' — then reuse it. 'Is this result ever used?' — then delete it.

How expensive the question is depends entirely on the representation. In a syntax tree, 'where does this variable's value come from at this point?' requires searching backwards through the tree and reasoning about control flow that the tree only represents implicitly.

In SSA form the same question is a pointer dereference. Every value is assigned exactly once, so each use names its unique definition directly (Module 07).

So changing the IR does not merely make optimisations faster; it makes different optimisations practical. Analyses that would be quadratic searches become constant-time lookups, which moves them from 'theoretically possible' to 'run on every function'. **That is a much stronger claim than a constant-factor speedup, and it is why SSA displaced everything else.**

Representation	Good for	Bad at
Abstract syntax tree	Source-level checks, refactoring tools, and error messages — it still corresponds to what the user wrote.	Dataflow. Control flow is implicit in the tree structure, so any question about what reaches what requires reconstructing it.
Three-address code	Simple, linear, and close to what machines execute.	No explicit link from a use back to its definition.

Representation	Good for	Bad at
Control flow graph	Control flow is explicit , which is the precondition for dataflow analysis (Module 08).	Values still have to be searched for — the graph says where control goes, not where data comes from.
SSA	Def-use is $O(1)$, which makes most modern optimisation practical. The default for nearly every serious compiler.	Requires ϕ functions at merge points and a destruction pass before code generation (Module 07, Module 11).
Stack machine code	Compact and very easy to generate from a tree.	Hard to analyse — operands are implicit in stack position. The JVM and WebAssembly use it, and their optimisers reconstruct an SSA form internally.
Machine code	It runs.	Target-specific and nearly unanalysable — which is what makes decompilation hard.

Real compilers use several of these simultaneously and lower between them. Clang alone has an AST, LLVM IR, SelectionDAG, and MachineIR, each chosen because it suits a particular range of transformations. **The question is never 'which IR' but 'which IR for which phase'**, and the lowering points between them are themselves design decisions.

3 Compilation and interpretation

Approach	When translation happens	The trade
Ahead-of-time compilation	Entirely before running.	The best code, and no runtime information. The compiler must guess which branch is hot and which types occur.
Tree-walking interpreter	Never — the tree is walked.	By far the simplest to build , and 10 to 100× slower than compiled code. Entirely adequate for configuration and for build scripts.
Bytecode virtual machine	Source to bytecode once; bytecode interpreted.	Portable, compact, and much faster than tree walking. The usual choice for an embedded scripting language (Module 12).
Just-in-time compilation	At runtime, for hot code only.	Uses real profiles — actual branch frequencies and actual types — and pays a warmup cost plus the memory for the compiler itself.
Tiered JIT	Progressively: interpret, then cheaply compile, then optimise hard.	Fast startup and fast peak performance. What V8 and HotSpot do, and the reason both are enormous.
Shader compilation	Partly offline to SPIR-V, partly in the driver at load or draw time.	Module 13 , and the direct cause of shader compilation stutter.

A JIT can beat an ahead-of-time compiler, which consistently surprises people. It knows which branch actually ran, which types actually occurred at a polymorphic call site, and which values turned out to be constant in practice — so it can specialise on facts that are true of this execution and are not provable in general. It guards the assumption and deoptimises when it fails, which is a trade an AOT compiler cannot make.

4 Why correctness dominates this course

A MISCOMPILE IS INVISIBLE FROM THE SOURCE

Every other program's bugs are explicable by reading it. The behaviour follows from the code, so the code is where you look. A compiler bug breaks that assumption entirely: **the source is correct and the behaviour is wrong**, and nothing in the program accounts for it.

So the search goes everywhere except where the problem is. Days are routinely lost before anyone seriously suspects the compiler, partly because the hypothesis feels like arrogance — the compiler is used by millions and your code is not.

And the blast radius is everything the compiler built. A single wrong optimisation silently corrupts every program that happened to trigger it, across every project using that compiler version, with no error anywhere.

Which is why Module 13's differential testing is not an appendix. Csmith — a random C program generator — found over 400 bugs in GCC and LLVM, production compilers that were already extremely heavily tested and used by essentially everyone. **Hand-written test suites did not find those bugs and would not have.**

UNDEFINED BEHAVIOUR IS WHERE MOST OF THEM LIVE

A language specification says what *must* happen and deliberately leaves the rest undefined — signed integer overflow, dereferencing null, data races, reading past an array, strict aliasing violations. Undefined means the standard imposes no requirement at all.

Optimisers exploit undefined behaviour aggressively, because assuming it cannot occur licenses much stronger transformations. If signed overflow is undefined, the compiler may assume $x + 1 > x$ always holds and delete the check.

So a program containing undefined behaviour can change meaning under optimisation — correctly, by the standard, and catastrophically in practice. **The canonical case is a null check deleted because the pointer was already dereferenced above it:** the dereference would be undefined if the pointer were null, so the compiler concludes it is not null, so the check is dead. Every step is valid; the result is a security hole.

This is not a compiler bug, and it is indistinguishable from one when you first meet it — the symptom is the same: correct-looking source, wrong behaviour, appears only at -O2. **Learning to tell the two apart is part of what this course is for**, and the practical tools are the sanitisers (CSCE 735 Module 13) and bisecting the optimisation level.

- **A language with good tagged unions** — Rust enums, C++ `std::variant` with a visitor, or OCaml. You will write a great deal of tree and graph manipulation, and the ergonomics of that one feature dominate the experience.
- **LLVM installed.** Run `clang -S -emit-llvm` on something small and read the output — **you are looking at the SSA form of Module 07**, and seeing it early makes that module land.
- **Read Crafting Interpreters, part II.** Having a complete working implementation in your head makes every later module concrete rather than abstract, and it is a weekend.

- **A SPIR-V toolchain** — glslangValidator and spirv-dis. **Disassemble one of your own shaders in week one**; it is the most motivating thing available in this course, and it is also an SSA IR you can read.
- **Pick your source language now and keep it deliberately small.** Integers, booleans, functions, control flow, and one aggregate type is plenty. **Scope creep in the source language is the usual way this project fails** — every feature added costs work in all eight phases.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Nystrom — Crafting Interpreters (free book)	Read part II this week. A complete tree-walking interpreter, then a complete bytecode VM, with every line explained.
Stanford CS143 — Compilers, lecture 1 (free)	The phase structure of §1, with worked examples of what each one catches.
Chris Lattner — LLVM, in The Architecture of Open Source Applications (free)	The §1 reuse argument and the §2 IR argument , from the person who built the system that made both famous. Short.
Yang, Chen, Eide & Regehr — Finding and Understanding Bugs in C Compilers (free)	The Csmith paper of §4. Read the results section now and the method in Module 13.
Regehr — A Guide to Undefined Behavior in C and C++ (free)	The §4 hazard, with the deleted-null-check example worked through.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Run `clang -S -emit-llvm` on a ten-line function and read the output. Identify the basic blocks.
- 2 Compile the same function at `-O0` and `-O2` and diff the IR. Name three transformations you can see.
- 3 **Disassemble one of your own shaders** with `spirv-dis` and find the entry point and the control flow.
- 4 Write a tokeniser for arithmetic expressions by hand, with no libraries.
- 5 Write a tree-walking interpreter for arithmetic with variables.
- 6 **Measure it against compiled C** on the same computation and report the ratio.
- 7 **Write a program with signed overflow** and show that it behaves differently at `-O0` and `-O2`.
- 8 Reproduce the deleted-null-check example and inspect the generated code.
- 9 Run the same program under UBSan and report what it says.

- 10 Specify your project language in one page** — grammar, types, and an explicit list of what it will not have.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the eight phases and say what each establishes.
- 2 Why is optimisation the odd phase out?
- 3 What does the front end/back end split buy, and what does it cost?
- 4 Why is the IR the central design decision?
- 5 Compare six representations on what they are good and bad at.
- 6 Give six points on the compile–interpret spectrum.
- 7 Why can a JIT beat an ahead-of-time compiler?
- 8 Why is a miscompile uniquely expensive to diagnose?
- 9 What is undefined behaviour and why do optimisers exploit it?